Query Performance Prediction Is Not Benefit Prediction: Observability for Selective Retrieval Interventions

Anonymous Author(s)

Abstract

Query performance prediction (QPP) estimates whether a ranking is poor, but selective retrieval needs a different prediction: whether an intervention will improve it. We formalize this distinction as a decision under observable telemetry and derive an exact, cost-aware decomposition of the gain that no policy intervening only on failing queries can reach. We evaluate LLM query expansion, cross-encoder reranking, and pseudo-relevance feedback on six benchmarks, four of them pre-registered and untouched. Without a budget cap, a perfect failure gate recovers only 8–57% of the oracle's gain. On the same queries, the learned failure gate's score ranks failing queries far better than queries the intervention helps (pre-registered on three untouched test sets: AUC 0.72–0.89 against 0.43–0.64). For fixed-candidate reranking, rescuing a failing query requires a judged-relevant document below the top ten by construction; yet the failure score often ranks such repairable failures below unrepairable ones (AUC 0.27–0.53; 0.37 and 0.40 pre-registered). The routing value of failure scores varies by corpus: clear on HotpotQA, against our pre-registered prediction, though computing the gate there costs more than reranking every query; small or uncertain elsewhere. Stronger pre-action predictors and partial probes help inconsistently. Selective retrieval should be evaluated by intervention benefit, routing gain, and computation cost, not by failure-detection accuracy alone.

1 Introduction

Retrieval interventions help some queries and harm others. LLM query expansion (QE) can rewrite a query using the evidence it retrieves [16], yet it harms a measurable fraction of queries [1, 30]; cross-encoder reranking and pseudo-relevance feedback (PRF) are equally uneven. The natural response is to intervene selectively, and the natural gate is a query performance predictor (QPP): intervene when the current ranking looks like it is failing [2, 12]. That the gate's target differs from the quantity a policy needs is not new: selective-QE work predicts expansion benefit directly [7], and uplift modelling shows that targeting high-risk cases is not targeting the cases a treatment helps [3, 9]. What is missing is how much a failure gate gives up, why, and whether the telemetry a deployed system can observe, its *observability*, can do better.

Contributions. (C1) A decision framing of selective intervention under observable telemetry, with an exact decomposition of the gain no policy acting only on failing queries can capture, at zero and positive cost (Section 3). **(C2)** Evidence on six corpora, four pre-registered and untouched, that the targets differ and that the difference has measurable consequences: the uncapped *failure-only ceiling* is 8–57% of the oracle's gain, and failure scores rank helped queries far below how they rank failing ones. **(C3)** A structural constraint, reachability, with evidence that failure scores often rank repairable failures below unrepairable ones, and pre-registered tests showing that the routing value of failure scores, stronger predictors, and probes varies by corpus.

2 Related Work

QPP and selective QE. Pre- and post-retrieval QPP (Clarity [6], WIG [34], NQC [24]) predicts query difficulty. Cronen-Townsend et al. [7] compare the clarity of expanded and unexpanded rankings; Amati et al. [2] gate expansion on difficulty predictors; Lv and Zhai [17] predict a feedback weight; Collins-Thompson [5] trades reward against risk. Even idealised selective QE needs QPP at Kendall $\tau \geq 0.5$ [11], and Chifu et al. [4] find QPP accuracy strongly collection-dependent and QPP-driven selective processing only marginally better. To our knowledge, no such work bounds what a failure-restricted policy can recover, scores failure and benefit on the same queries, or explains the gate's selection error.

LLM QE and adaptive retrieval. Query2doc [29], MuGI [33], and CSQE [16] expand unconditionally; generative expansions often hurt strong retrievers [30] and fail on unfamiliar or ambiguous queries [1]. Kotte [13] gates prompt-only rewriting for dense retrievers without reliable gain. Sarode [22] finds that a learned selector over candidate LLM rewrites captures little oracle headroom and transfers inconsistently; we ask whether to intervene at all, across three interventions, and bound failure-restricted policies. Adaptive-RAG [12] routes on question complexity and Mallen et al. [20] retrieve only for less popular entities. Graph-based adaptive reranking [18] extends the candidate pool beyond first-stage retrieval; we instead ask whether failure scores identify which queries a fixed-pool intervention can help. Cascade rankers [28] spend costly stages only on promising candidates, and cutoff and efficiency predictors [8, 26] set per-query reranking depth or pruning. In uplift terms [9], both outcomes are observed offline, so our label is exact up to relevance noise.

3 Framework

For a query q, let R_0 be the original ranking and R_a the ranking after intervention a, with metric M (nDCG@10). The intervention's effect is $\Delta_a(q) = M(R_a, q) - M(R_0, q)$ and its cost c_a ($\Delta_0 = c_0 = 0$). With full information the best decision is $a^*(q) = \arg\max_a\{\Delta_a(q) - \kappa c_a\}$, for a cost weight κ. A deployed controller sees only telemetry Z, so the best it can do is $\pi^*(Z) = \arg\max_a\{\mathbb{E}[\Delta_a \mid Z] - \kappa c_a\}$. What matters is therefore what Z reveals about Δ_a, not about $M(R_0)$. Two information sets differ in cost: *pre-action* telemetry Z_0 describes R_0 and costs no intervention, though computing it is not free; *post-action* telemetry Z_1 also describes R_a and exists only after c_a is paid. A *probe* observes part of the intervention, such as cross-encoder scores of a few candidates, at part of its cost (Section 9). A failure gate replaces $\mathbb{E}[\Delta_a \mid Z_0]$ with $P(F \mid Z_0)$, where $F = \mathbf{1}[M(R_0, q) = 0]$.

The failure-only ceiling. Fix one intervention with $c_a = 1$, so κ is its cost per query in nDCG@10 units. The oracle selector

earns $G_{\text{orc}}(\kappa) = \mathbb{E}[(\Delta - \kappa)_+]$. A policy that may act only on failing queries does best by intervening where $F = 1$ and $\Delta > \kappa$, earning $G_F^*(\kappa) = \mathbb{E}[F(\Delta - \kappa)_+]$ (the *failure-restricted oracle*; its share of G_{orc} is the failure-only ceiling), so

$$G_{\text{orc}}(\kappa) - G_F^*(\kappa) = \mathbb{E}[(1 - F)(\Delta - \kappa)_+]$$

(gain on healthy queries) bounds every failure-restricted policy at any $\kappa \geq 0$. Because $M \geq 0$, a failing query cannot be harmed ($\Delta \geq 0$ when $F = 1$), so at $\kappa = 0$ a *perfect* failure gate, intervening iff $F = 1$, attains G_F^*. At $\kappa > 0$ it earns $G_F(\kappa) = \mathbb{E}[F(\Delta - \kappa)]$, a further $\mathbb{E}[F(\kappa - \Delta)_+]$ below G_F^* spent on failing queries helped by less than κ, which only information beyond the binary label avoids. The identity is elementary; its use is that every term is measurable offline for any intervention. It does not bound a policy built from a failure *score* that also acts on healthy queries.

Information vs. policy regret. For a policy $\pi(S)$ on a score S at budget b, $V_{\text{orc}} - V_\pi = (V_{\text{orc}} - V_S^*) + (V_S^* - V_\pi)$, with V_S^* the best budget-feasible policy using S alone: lost information plus misuse. We approximate it with cross-fitted ten-bin policies and report an optimistic in-sample optimum in the same class (Section 10); neither bounds V_S^*.

4 Setup

Data. TripClick [21]: 1.52M health documents, the TAIL bucket's validation and test splits (1,175 queries each), with click-derived relevance labels. BEIR NQ [14, 25]: 2.68M passages, human labels; 800 validation and a disjoint 2,052-query test set. BEIR ArguAna [27]: 8,674 arguments, 1,406 queries whose relevant document is the counterargument; BEIR FiQA [19]: 57,638 financial posts; BEIR DBPedia-Entity [10]: 4.63M entities, 400 test queries with graded labels; BEIR HotpotQA [32]: 5.23M passages, 7,405 multi-hop test questions (Section 8).

Interventions. *LLM QE*: BM25 over SQLite FTS5; a deterministic rule picks one of four expansion operators or none, and `qwen2.5:3b` (4-bit, greedy, ≤ 2 calls) writes a corpus-grounded expansion. On TripClick and under dense retrieval (`bge-small-en-v1.5` [31]) the expanded ranking's top candidates are reranked by a MiniLM cross-encoder [23]; the original is not, so those labels include the reranker's effect. TripClick's deployed QE fuses original and expanded rankings (RRF); NQ's uses the expanded ranking. *Reranking*: the cross-encoder scores the BM25 top 100; its top 50 lead. *PRF*: RM3-style [15] weights from the top d documents select t terms whose BM25 ranking is interpolated with the original (weight λ); (d, t, λ) is tuned on validation over 108 settings.

Labels and models. *Failure*: $F = 1$ (45% of TripClick and 49% of NQ test queries under BM25). *Harmed*: $\Delta < 0$, ties not harmed. Detection is a logistic regression on 8 pre-action features (score shape, term coverage, rare-term IDF, entity overlap, coherence, perturbation stability; 6 under dense). Action utility is fit on Δ from post-action features. For pre-action routing, a *benefit* model regresses Δ on the 8 pre-action features alone. Three further benefit predictors, pre-registered for ArguAna (Section 8), use 10 pre-action features (the 8 above, NQC, query length), as does the failure gate on ArguAna and FiQA: B1, LightGBM regression on Δ; B3, LightGBM quantile regression; and B2, the distance-weighted mean Δ of the k nearest training queries under frozen `bge-small-en-v1.5` embeddings. Every model and threshold is fit on validation and scored once on test; intervals are paired bootstraps over queries (10^4 resamples). We call the deployed detector, used to route, the *learned failure gate*, and any model of Δ a *benefit predictor*.

Table 1: Failure-only ceiling (test). Rescue: $P(\Delta > 0 \mid F)$. Healthy helped / harmed: $P(\Delta \gtrless 0 \mid \neg F)$. Share: perfect failure gate's gain over the oracle's at $\kappa = 0$, 95% CI. Below the first rule: pre-registered, untouched test (n = 430, 648, 400, 7,405).

Setting	$P(F)$	Rescue	Healthy help	harm	Share
LLM QE, TripClick	.45	.07	.15	.19	42% [32, 52]
LLM QE, NQ	.49	.07	.19	.33	33% [26, 40]
LLM QE, TripClick, dense	.59	.06	.25	.41	34% [25, 43]
LLM QE, NQ, dense	.26	.15	.19	.29	33% [27, 40]
Reranking, TripClick	.45	.17	.24	.56	52% [44, 60]
Reranking, NQ	.49	.46	.55	.19	57% [54, 60]
PRF, TripClick	.45	.01	.12	.20	8% [1, 17]
PRF, NQ	.49	.05	.20	.22	32% [25, 39]
Reranking, ArguAna	.21	.36	.25	.53	38% [27, 49]
PRF, ArguAna	.21	.01	.05	.06	18% [0, 52]
LLM QE, ArguAna	.21	.27	.23	.59	39% [27, 52]
Reranking, FiQA	.51	.29	.52	.23	46% [39, 53]
PRF, FiQA	.51	.02	.14	.19	28% [11, 46]
Reranking, DBPedia	.22	.58	.77	.17	21% [15, 28]
Reranking, HotpotQA	.11	.61	.43	.11	25% [23, 27]

Evidence ledger. Only the ArguAna, FiQA, DBPedia, and HotpotQA results (Section 8) are independent confirmation: every NQ query and the TripClick TAIL test set have been inspected. *Fixed before a test run*: the NQ, reranking, and PRF runs. *Pre-committed analyses of inspected traces*: the detection AUCs, ceiling, budget routing, and Holm correction. *Exploratory*: TripClick test was used during development, and the NQ B1/B2 runs, diagnostics, probes of Section 9, and the reachability analysis on the first four corpora followed protocols frozen before computing but after the test sets were inspected; analyses added after review are marked in the text. Pre-commitments and scripts are in the anonymized repository: https://anonymous.4open.science/r/agentic-query-expansion-8939/.

5 The Failure-Only Ceiling

Failure-restricted policies miss most of the gain. The failure-restricted oracle captures 8–57% of the oracle's gain (Table 1; 18–46% on ArguAna and FiQA), though it never makes a query worse; the lowest shares, and the widest intervals, come from PRF, which barely acts. The share follows how often an intervention rescues a failing query relative to how often it helps a healthy one: reranking rescues 17–46% of failing queries, so up to half its headroom lies there; LLM QE rescues 6–15% and PRF almost never. The rest sits in healthy queries the intervention also often harms, which only benefit prediction can separate. At a per-query cost of 0.02 the shares are similar (8–57%; 22–47% on ArguAna and FiQA), and intervening on every failing query then loses outright for PRF and dense TripClick QE.

A budget cap loosens the restriction only where rescues are common (exploratory, post-review). At a 10% cap, failing queries hold 54–98% of the capped oracle's gain for reranking and ArguAna QE (83% on FiQA); the other settings stay near their uncapped shares.

Table 2: One score, two targets (test). Failure: AUC of the learned failure gate's score for F. Helped: its AUC for $\Delta > 0$ on the same queries, 95% CI. Within failing: helped AUC among failing queries (–: fewer than 20 rescued). Failure is shared by the interventions on a corpus and retriever. Exploratory except the last four rows (pre-registered).

Setting	Failure	Helped	Within failing
LLM QE, TripClick	0.65	0.52 [0.46, 0.57]	0.47
LLM QE, NQ	0.68	0.50 [0.46, 0.53]	0.44
LLM QE, TripClick, dense	0.64	0.50 [0.45, 0.54]	0.51
LLM QE, NQ, dense	0.69	0.56 [0.53, 0.59]	0.50
Reranking, TripClick	0.65	0.47 [0.43, 0.51]	0.44
Reranking, NQ	0.68	0.48 [0.46, 0.51]	0.37
PRF, TripClick	0.65	0.49 [0.42, 0.55]	–
PRF, NQ	0.68	0.45 [0.41, 0.48]	0.34
Reranking, ArguAna	0.65	0.43 [0.37, 0.49]	0.34
PRF, ArguAna	0.65	0.54 [0.39, 0.68]	–
LLM QE, ArguAna	0.65	0.42 [0.36, 0.48]	0.36
Reranking, FiQA	0.72	0.52 [0.47, 0.57]	0.56
PRF, FiQA	0.72	0.46 [0.37, 0.54]	–
Reranking, DBPedia	0.76	0.43 [0.36, 0.49]	0.48
Reranking, HotpotQA	0.89	0.64 [0.63, 0.66]	0.37

The learned failure gate also selects poorly. At a 10% budget, 35–80% of its picks are failing queries (the rest are healthy), and it realises at most 34% of the capped failure-restricted oracle's gain, and its gain is negative in five of the first eleven settings (all three on ArguAna) and in 7–9 of them at 25–75%. Among failing queries its score ranks the rescued ones no better than chance (Table 2; below it in four settings, not on FiQA).

The ceiling and the selection error are separate constraints, and both operate at every budget below 100%.

6 Detection vs. Benefit

The failure gate detects failure, not benefit. The learned failure gate ranks failing queries at AUC 0.64–0.69 (Table 2), ordinary QPP accuracy: NQC [24] reaches 0.647 on TripClick and 0.721 on NQ. Scored against whether the intervention helps (exploratory, post-review), the same scores reach only 0.42–0.56, with intervals below 0.5 for NQ PRF and ArguAna QE and reranking and above only for dense NQ QE. Pre-registered on FiQA, DBPedia, and HotpotQA, the gap replicates in every setting (Holm $p < 0.001$; Section 8), and Figure 1 shows its shape.

Benefit predictors do better but are weak (helped AUC $\leq$ 0.58; NQ probe 0.65, HotpotQA B1 0.67 and probe 0.73). They beat the failure score on NQ reranking (+0.08 to +0.17), NQ PRF, and ArguAna QE and reranking (+0.07 to +0.12); elsewhere differences mostly span zero.

Why: reachability (exploratory on the first four corpora, protocol frozen before computing; pre-registered on DBPedia and HotpotQA). For fixed-candidate reranking, rescuing a failing query requires a judged-relevant document at BM25 ranks 11–100, by construction. Empirically, such *reachable* failures vary widely (51–83% of failing queries), 29–88% of them are rescued, yet the failure score often ranks reachable failures below unreachable ones (AUC 0.27–0.53; 0.40 [0.27, 0.54] and 0.37 [0.33, 0.41] pre-registered), and among reachable failures the helped rate shows no consistent trend across its tertiles. The highest failure scores can thus prioritise queries this reranker cannot repair. Among healthy queries, harm concentrates where the first relevant document is already at rank 1 (mean Δ −0.02 to −0.35 on the first four corpora).

Table 3: Pre-registered tests on DBPedia and HotpotQA (reranking, test; one Holm family per corpus). S: learned failure gate's score. P2–P4 spend each budget (10–75%, averaged) in order of $\mathbb{E}[\Delta]$ in ten score bins, cross-fitted over test queries (5 folds, 20 repeats): a test of the information in a score, not a deployable policy. Bold: against the prediction.

Test (prediction)	DBPedia	HotpotQA
P1 $\mathrm{AUC}_F - \mathrm{AUC}_{\Delta>0}$ (> 0)	+0.33 [0.24, 0.42]	+0.24 [0.23, 0.26]
P2 S − random (n.s.)	+0.001 [−0.002, 0.013]	**+0.023** [0.021, 0.025]
P3 B1 − S (n.s.)	+0.003 [−0.009, 0.011]	**+0.003** [0.002, 0.005]
P4 Probe-10 − S (> 0)	**+0.002** [−0.009, 0.011]	+0.008 [0.007, 0.010]
P5 AUC reachable (≤ 0.6)	0.40 [0.27, 0.54]	0.37 [0.33, 0.41]

7 Routing at Matched Budgets

Pre-action routing. At every budget (10–75%), each scorer that decides before intervening stays close to random routing and far below the oracle on TripClick, NQ, and ArguAna (table and cost curves in the repository). Failure routing beats random by at most 0.006 at any budget and often falls below it. Averaged over budgets, NQC is no better than the detector, and benefit routing beats failure routing only for reranking on NQ (+0.009 [+0.003, +0.015]), where it leads at every budget (per-budget CIs in the repository); B1 and B2 do no better (exploratory).

8 Pre-Registered Replications

ArguAna. *Question:* do stronger pre-action benefit predictors beat failure routing on data never inspected for this work? *Design*, pre-registered before download (`configs/arguana_precommit.json`): a debate-grouped split (701/275/430), the three interventions (LLM QE as on NQ), and Holm-corrected primary contrasts for B1–B3 (B3's registered quantile choice is unsound, so a secondary B3 fixes 0.25); leakage fixes were dated amendments before any label was read. *Result: no.* Every intervention loses on average (reranking −0.071 [−0.108, −0.034], LLM QE −0.163), yet the oracle could gain 0.053 and 0.010 budget-averaged, so the task is pure selection. For reranking, B1 minus failure routing is +0.015 [−0.001, +0.028] (Holm $p = 0.13$), B2 minus failure +0.008 (Holm $p = 0.37$), B3 minus B1 under a budget cap +0.006 ($p = 0.52$); the best recovers 16% of the oracle-minus-failure headroom, below the pre-registered 25% bar, and a secondary family agrees (none of nine survives Holm). Failure routing of reranking is below random at every budget. PRF changes only 9% of rankings, so its rows are uninformative. At 430 queries, B1's +0.015 may be real but is unresolved.

FiQA (predictions fixed before download, one Holm family; 1,500/500/648 queries). *Confirmed:* the failure score ranks failure above benefit (0.72 against 0.52 and 0.46, Holm $p < 0.001$). *Not confirmed:* the within-failing inversion (0.56 [0.49, 0.63]) and Probe-10 beating failure and B1 routing (−0.001, +0.006; Holm $p \geq 0.25$); B1 and B2 do not beat failure routing either (−0.006 each).

DBPedia and HotpotQA. *Question:* on new data, does the failure score carry routing information, and is any shortfall specific to

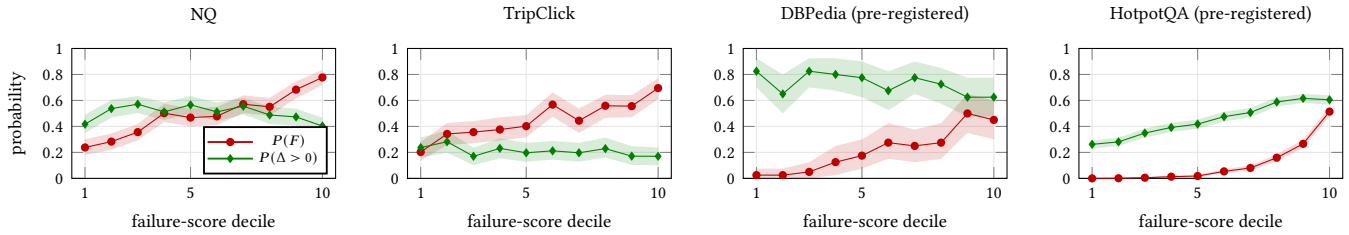


Figure 1: One score, two targets (reranking, test, 95% bootstrap bands). By decile of the learned failure gate's score, the failure rate rises steeply; the share of queries reranking helps stays flat or falls on NQ, TripClick, and DBPedia but rises on HotpotQA. NQ, TripClick: exploratory.

QPP? *Predictions*, fixed before download (`configs/info_value_precommit.json`), were built to let the account 'pre-action scores carry little routing information' fail: it fails if routing on S beats random (P2) or B1 beats S (P3); reachability fails if S separates reachable from unreachable failing queries (P5). DBPedia's 400 test queries are scored by five-fold cross-fitting; HotpotQA uses 1,500 training and 500 dev queries. *Result* (Table 3): the detection–benefit gap (P1) and reachability (P5) hold on both. DBPedia behaves as predicted, except that the probe does not help. On HotpotQA, against the prediction, S beats random by +0.023, and B1 (+0.003, small) and the probe beat S (Holm $p < 0.001$); simply ranking by S gives the same +0.024 (exploratory). There, reranking helps more the higher the failure score (Figure 1), though not within failing queries (P5). Mean gain decomposes exactly as $m = p_F p_R \mu_R + (1 - p_F)\mu_H$, with p_F the failure rate, p_R reachability among failures, and μ_R, μ_H the mean Δ of reachable failures and healthy queries. A four-factor Shapley attribution of the change in m from the lowest to the highest S tertile (+0.17) gives μ_H +0.12 [0.11, 0.13], p_F +0.06, μ_R +0.01, p_R −0.01; on NQ, p_R (−0.08) cancels p_F (+0.05) (descriptive). HotpotQA's two-passage judgments may contribute, untested. The gate's telemetry costs 2.8 s per query beyond the shared BM25 retrieval, reranking 0.22 s, so reranking every query is cheaper; but a gate using only BM25 score-list features and query length (0.3 ms per query) matches S (paired difference +0.001 [−0.0002, +0.003]) and keeps 75% of always reranking's gain at half its cost (exploratory). PRF barely acts (mean Δ ±0.003).

9 Partial Observation and Cost

A probe sits between pre- and post-action telemetry. In an exploratory analysis (protocol frozen before computing), the cross-encoder scores the first 10 BM25 candidates; B1's model regresses full reranking's Δ on nine probe features, and the top-scoring queries are fully reranked. Each policy is charged its median latency on 50 validation queries: full reranking 0.48, 1.11, 1.26, 0.84 s on NQ, TripClick, ArguAna, FiQA; the probe 0.05–0.13 s.

On NQ, where reranking usually helps, the Probe-10 policy beats failure routing by +0.024 [+0.017, +0.031] and B1 by +0.017 [+0.011, +0.022] over 10–75% reranked (both Holm-significant), though NQ's pre-action telemetry costs more than reranking every query; against free NQC (post-review), probe routing gains +0.020 [+0.012, +0.028] at equal median-latency cost. Probes do not help on TripClick and ArguAna, where reranking loses on average, nor on FiQA or DBPedia; they do on HotpotQA (Section 8).

10 Discussion and Limitations

QPP and selective retrieval predict different targets, and the difference matters. One consequence is proven: at any cost, failure-restricted policies are capped by the oracle gain on failing queries (8–57% uncapped). The other is empirical, replicated on three untouched corpora: failure scores rank helped queries far below failing ones, and for reranking they often rank repairable failures last. Yet failure routing has heterogeneous value: raw routing on the failure score beats random clearly on HotpotQA (+0.024), slightly on FiQA (+0.008 [+0.002, +0.013]), and not detectably on DBPedia (+0.002 [−0.004, +0.007]; all exploratory), whose 400 queries cannot exclude an effect a third of HotpotQA's. High QPP accuracy neither guarantees nor precludes intervention value; benefit alignment and the cost of the signal decide. Most of what failure routing loses is not recovered by a better rule: within the ten-bin policy class, calibration adds little over ranking by S (except by abstaining on ArguAna), and even an optimistic in-sample policy leaves 65–93% of the oracle–random gap. This does not bound arbitrary policies on the continuous score. Reachability is predictable from deep BM25 scores (AUC 0.59–0.70 on five corpora), yet a reachability-factored benefit model does not beat a matched single model on any corpus (Holm $p = 1.0$; all exploratory).

Limitations. One small (3B) generator; reranking and PRF only under BM25, with one cross-encoder and one PRF family. Six corpora, four untouched; three (430, 648, 400 test queries) are underpowered for differences below about 0.015; under dense retrieval, retriever strength and domain are confounded, and the dense QE labels include the reranker. TripClick's labels are click-derived and sparse. The cost weight κ is illustrative, latencies are single-machine (median, P95 check), and probes cover reranking only. All pre-committed results are reported.

11 Conclusion

QPP predicts whether a ranking is poor; selective retrieval needs to know whether an intervention helps. We bound the gain failure-restricted policies cannot reach and show that learned failure scores often misrank intervention opportunities. For fixed-pool reranking, reachability explains part of the mismatch: severe failures can be impossible to rescue. The value of failure-based routing still varies by corpus, as the pre-registered HotpotQA result shows. Selective retrieval should be evaluated by intervention benefit, routing gain, and computation cost, not by failure-detection accuracy alone.

Ethical Considerations

All data (TripClick and BEIR NQ, ArguAna, FiQA, DBPedia-Entity, and HotpotQA) are existing research benchmarks used under their licenses; TripClick is used for non-commercial research, and no model or per-query output fit on it is released. TripClick's click logs are aggregate, de-identified interactions, and we make no attempt to re-identify searchers. No human-subjects data was collected. The generator is a small, locally hosted model used only to rewrite queries within a bounded action space. Our central finding cautions against deploying generative interventions on the strength of a failure detector alone.

References

[1] Kenya Abe, Kunihiro Takeoka, Makoto P. Kato, and Masafumi Oyamada. 2025. LLM-based Query Expansion Fails for Unfamiliar and Ambiguous Queries. *arXiv preprint arXiv:2505.12694* (2025). https://arxiv.org/abs/2505.12694

[2] Giambattista Amati, Claudio Carpineto, and Giovanni Romano. 2004. Query Difficulty, Robustness, and Selective Application of Query Expansion. In *Advances in Information Retrieval: 26th European Conference on IR Research (ECIR) (LNCS, Vol. 2997)*. 127–137. doi:10.1007/978-3-540-24752-4_10

[3] Eva Ascarza. 2018. Retention Futility: Targeting High-Risk Customers Might Be Ineffective. *Journal of Marketing Research* 55, 1 (2018), 80–98. doi:10.1509/jmr.16.0163

[4] Adrian-Gabriel Chifu, Sébastien Déjean, Josiane Mothe, Moncef Garouani, Diego Ortiz, and Md Zia Ullah. 2026. Uncovering the Limitations of Query Performance Prediction: Failures, Insights, and Implications for Selective Query Processing. *ACM Transactions on Information Systems* (2026). doi:10.1145/3774427

[5] Kevyn Collins-Thompson. 2009. Reducing the Risk of Query Expansion via Robust Constrained Optimization. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management (CIKM)*. 837–846. doi:10.1145/1645953.1646059

[6] Steve Cronen-Townsend, Yun Zhou, and W. Bruce Croft. 2002. Predicting Query Performance. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 299–306. doi:10.1145/564376.564429

[7] Steve Cronen-Townsend, Yun Zhou, and W. Bruce Croft. 2004. A Framework for Selective Query Expansion. In *Proceedings of the Thirteenth ACM International Conference on Information and Knowledge Management*. 236–237. doi:10.1145/1031171.1031220

[8] J. Shane Culpepper, Charles L. A. Clarke, and Jimmy Lin. 2016. Dynamic Cutoff Prediction in Multi-Stage Retrieval Systems. In *Proceedings of the 21st Australasian Document Computing Symposium (ADCS)*. 17–24.

[9] Carlos Fernández-Loría and Foster Provost. 2022. Causal Classification: Treatment Effect Estimation vs. Outcome Prediction. *Journal of Machine Learning Research* 23, 59 (2022), 1–35.

[10] Faegheh Hasibi, Fedor Nikolaev, Chenyan Xiong, Krisztian Balog, Svein Erik Bratsberg, Alexander Kotov, and Jamie Callan. 2017. DBpedia-Entity v2: A Test Collection for Entity Search. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1265–1268. doi:10.1145/3077136.3080751

[11] Claudia Hauff and Leif Azzopardi. 2009. When is Query Performance Prediction Effective?. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 829–830. doi:10.1145/1571941.1572150

[12] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*. https://aclanthology.org/2024.naacl-long.389/

[13] Varun Kotte. 2026. Not All Queries Need Rewriting: When Prompt-Only LLM Refinement Helps and Hurts Dense Retrieval. *arXiv preprint arXiv:2603.13301* (2026). https://arxiv.org/abs/2603.13301

[14] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, et al. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics* 7 (2019), 452–466. doi:10.1162/tacl_a_00276

[15] Victor Lavrenko and W. Bruce Croft. 2001. Relevance-Based Language Models. In *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 120–127. doi:10.1145/383952.383972

[16] Yibin Lei, Yu Cao, Tianyi Zhou, Tao Shen, and Andrew Yates. 2024. Corpus-Steered Query Expansion with Large Language Models. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*. 393–401.

[17] Yuanhua Lv and ChengXiang Zhai. 2009. Adaptive Relevance Feedback in Information Retrieval. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management (CIKM)*. 255–264. doi:10.1145/1645953.1645988

[18] Sean MacAvaney, Nicola Tonellotto, and Craig Macdonald. 2022. Adaptive Re-Ranking with a Corpus Graph. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management*. 1491–1500. doi:10.1145/3511808.3557231

[19] Macedo Maia, Siegfried Handschuh, André Freitas, Brian Davis, Ross McDermott, Manel Zarrouk, and Alexandra Balahur. 2018. WWW'18 Open Challenge: Financial Opinion Mining and Question Answering. In *Companion Proceedings of The Web Conference 2018*. 1941–1942. doi:10.1145/3184558.3192301

[20] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9802–9822. doi:10.18653/v1/2023.acl-long.546

[21] Navid Rekabsaz, Oleg Lesota, Markus Schedl, Jon Brassey, and Carsten Eickhoff. 2021. TripClick: The Log Files of a Large Health Web Search Engine. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2507–2513. doi:10.1145/3404835.3463242

[22] Gauri Sarode. 2026. Not Every Expansion Helps: The Selection and Generalization Gap in LLM Query Expansion. In *Proceedings of the 14th ACM IKDD International Conference on Data Science (CODS)*. To appear.

[23] Sentence Transformers. 2026. cross-encoder/ms-marco-MiniLM-L6-v2. https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2

[24] Anna Shtok, Oren Kurland, David Carmel, Fiana Raiber, and Gad Markovits. 2012. Predicting Query Performance by Query-Drift Estimation. *ACM Transactions on Information Systems* 30, 2 (2012), 11:1–11:35. doi:10.1145/2180868.2180873

[25] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*. https://arxiv.org/abs/2104.08663

[26] Nicola Tonellotto, Craig Macdonald, and Iadh Ounis. 2013. Efficient and Effective Retrieval Using Selective Pruning. In *Proceedings of the Sixth ACM International Conference on Web Search and Data Mining (WSDM)*. doi:10.1145/2433396.2433407

[27] Henning Wachsmuth, Shahbaz Syed, and Benno Stein. 2018. Retrieval of the Best Counterargument without Prior Topic Knowledge. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*. 241–251.

[28] Lidan Wang, Jimmy Lin, and Donald Metzler. 2011. A Cascade Ranking Model for Efficient Ranked Retrieval. In *Proceedings of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 105–114. doi:10.1145/2009916.2009934

[29] Liang Wang, Nan Yang, and Furu Wei. 2023. Query2doc: Query Expansion with Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 9414–9423. doi:10.18653/v1/2023.emnlp-main.585

[30] Orion Weller, Kyle Lo, David Wadden, Dawn Lawrie, Benjamin Van Durme, Arman Cohan, and Luca Soldaini. 2024. When do Generative Query and Document Expansions Fail? A Comprehensive Study Across Methods, Retrievers, and Datasets. In *Findings of the Association for Computational Linguistics: EACL 2024*. 1987–2003.

[31] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2023. C-Pack: Packed Resources For General Chinese Embeddings. *arXiv preprint arXiv:2309.07597* (2023). https://arxiv.org/abs/2309.07597

[32] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2369–2380. doi:10.18653/v1/D18-1259

[33] Le Zhang, Yihong Wu, Qian Yang, and Jian-Yun Nie. 2024. Exploring the Best Practices of Query Expansion with Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 1872–1883. doi:10.18653/v1/2024.findings-emnlp.103

[34] Yun Zhou and W. Bruce Croft. 2007. Query Performance Prediction in Web Search Environments. In *Proceedings of the 30th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 543–550. doi:10.1145/1277741.1277835